Отчёт по лабораторной работе №6

Дисциплина: Компьютерный практикум по статистическому анализу данных

Выполнил: Танрибергенов Эльдар (НПИбд-01-22)

Содержание

1 Цель работы **5**

2 Выполнение лабораторной работы **6**

2.1 Решение обыкновенных дифференциальных уравнений 6

2.1.1 Модель экспоненциального роста 6

2.1.2 Система Лоренца 9

2.2 Модель Лотки–Вольтерры 12

2.3 Задания для самостоятельного выполнения 14

3 Выводы **20**

Список иллюстраций

2.1 Численное решение модели экспоненциального роста 7
2.2 График: модель экспоненциального роста 8
2.3 Задание точности 8
2.4 График: модель экспоненциального роста с заданной точностью . 9
2.5 Система Лоренца 9
2.6 Численное решение системы Лоренца 10
2.7 График: Аттрактор Лоренца 11
2.8 График: Аттрактор Лоренца (интерполяция отключена) 12
2.9 Модель Лотки–Вольтерры 12
2.10 Численное решение модели Лотки–Вольтерры 13
2.11 Модель Лотки–Вольтерры: динамика изменения численности популяций 13
2.12 Модель Лотки–Вольтерры: фазовый портрет 14
2.13 Модель роста численности изолированной популяции (модель Мальтуса) 15
2.14 Модель Мальтуса: график 16
2.15 Логистическая модель роста популяции 16
2.16 Логистическая модель роста популяции: график 17
2.17 Модель эпидемии Кермака–Маккендрика (SIR-модель) 18
2.18 SIR-модель: график 19

Список таблиц

1 Цель работы

Основной целью работы является освоение специализированных пакетов для решения задач в непрерывном и дискретном времени.

2 Выполнение лабораторной работы

2.1 Решение обыкновенных дифференциальных уравнений

Обыкновенное дифференциальное уравнение (ОДУ) описывает изменение некоторой переменной u: $u'(t) = f(u(t), p, t)$, где $f(u(t), p, t)$ — нелинейная модель (функция) изменения $u(t)$ с заданным начальным значением $u(t_0) = u_0$, p — параметры модели, t — время. Для решения обыкновенных дифференциальных уравнений (ОДУ) в Julia можно использовать пакет diffrentialEquations.jl.

2.1.1 Модель экспоненциального роста

Рассмотрим пример использования этого пакета для решение уравнения модели экспоненциального роста, описываемую уравнением

$u'(t) = \alpha u(t), u(0) = u_0$, где α — коэффициент роста.Предположим, что заданы следующие начальные данные $\alpha = 0.98$, u(0) = 1,0, $t \in [0; 1,0]$.

Аналитическое решение модели имеет вид:

$u(t) = u_0 exp(\alpha t) u(t)$.

Численное решение в Julia будет иметь следующий вид:

```
[2]: # подключаем необходимые пакеты:
import Pkg
Pkg.add("DifferentialEquations")
using DifferentialEquations
# задаём описание модели с начальными условиями:
a = 0.98
f(u,p,t) = a*u
u0 = 1.0
# задаём интервал времени:
tspan = (0.0,1.0)
# решение:
prob = ODEProblem(f,u0,tspan)
sol = solve(prob)
```

```
   Resolving package versions...
     Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
    Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
```

```
[2]: retcode: Success
Interpolation: 3rd order Hermite
t: 5-element Vector{Float64}:
 0.0
 0.10042494449239292
 0.3521860297865888
 0.6934436122197829
 1.0
u: 5-element Vector{Float64}:
 1.0
 1.1034222047865465
 1.4121908713484919
 1.9730384457359198
 2.6644561424814266
```

Рис. 2.1: Численное решение модели экспоненциального роста

```
[3]: # подключаем необходимые пакеты:
Pkg.add("Plots")
using Plots
# строим графики:
plot(sol, linewidth=5,title="Модель экспоненциального роста", xaxis="Время",yaxis="u(t)",label="u(t)")
plot!(sol.t, t->1.0*exp(a*t),lw=3,ls=:dash,label="Аналитическое решение")
```

```
Resolving package versions...
  Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
 Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
```

[3]:

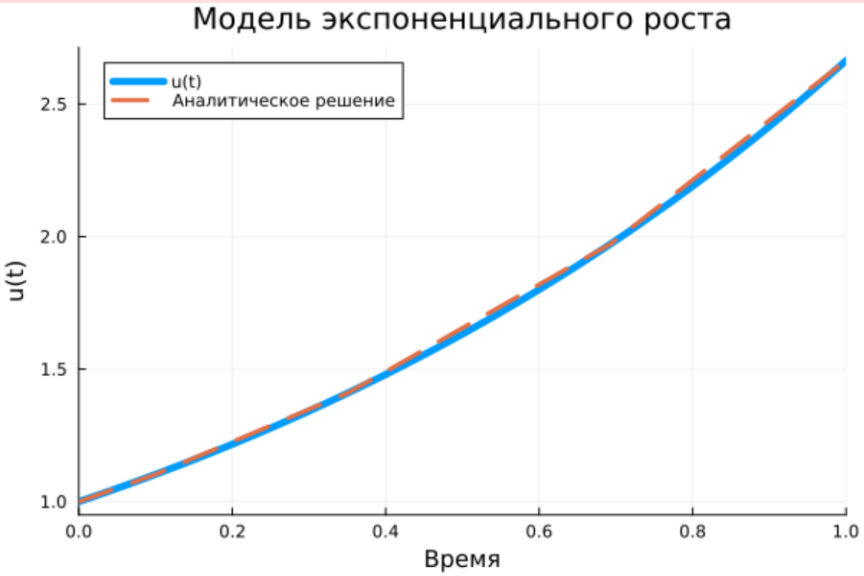

Рис. 2.2: График: модель экспоненциального роста

При построении одного из графиков использовался вызов sol.t, чтобы захватить массив моментов времени. Массив решений можно получить, воспользовавшись sol.u. Если требуется задать точность решения, то можно воспользоваться параметрами abstol (задаёт близость к нулю) и reltol (задаёт относительную точность).

```
[5]: # задаём точность решения:
sol = solve(prob,abstol=1e-8,reltol=1e-8)
println(sol)
# строим график:
plot(sol, lw=2, color="black", title="Модель экспоненциального роста", xaxis="Время",yaxis="u(t)",label="Численное решение")
plot!(sol.t, t->1.0*exp(a*t),lw=3,ls=:dash,color="red",label="Аналитическое решение")
```

```
ODESolution{Float64, 1, Vector{Float64}, Nothing, Nothing, Vector{Float64}, Vector{Vector{Float64}}, Nothing, ODEProblem{Flo
```

Рис. 2.3: Задание точности

[5]:

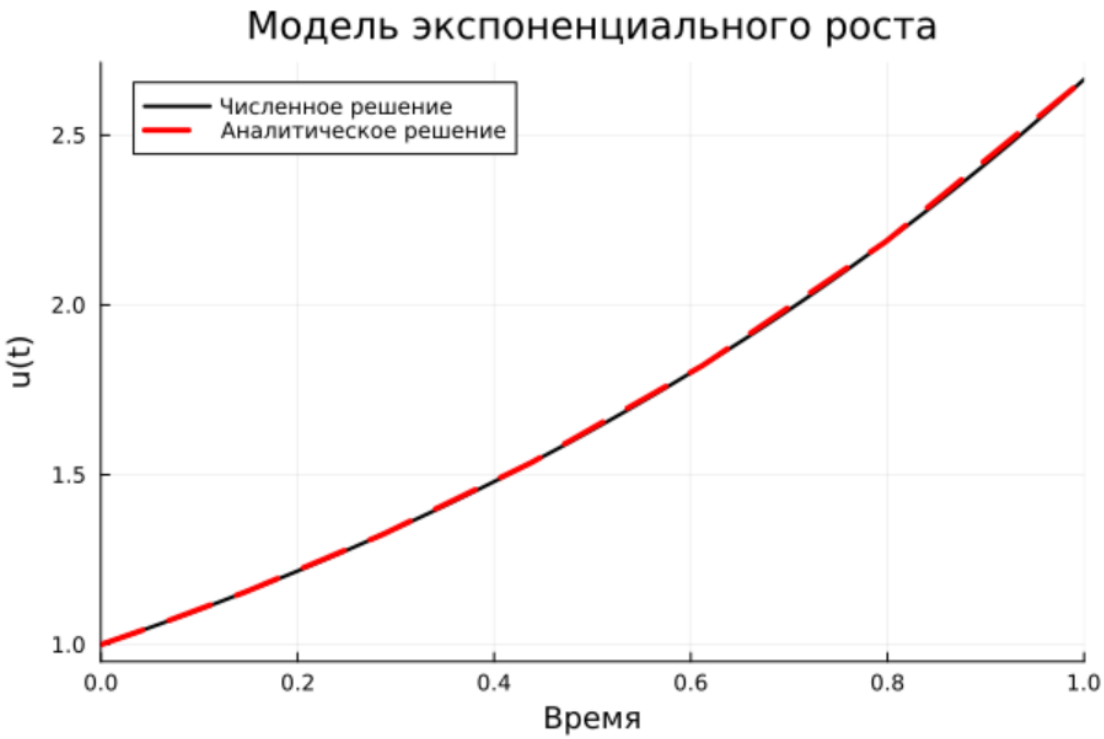

Рис. 2.4: График: модель экспоненциального роста с заданной точностью

2.1.2 Система Лоренца

Динамической системой Лоренца является нелинейная автономная система обыкновенных дифференциальных уравнений третьего порядка:

$$\begin{cases} \dot{x} = \sigma(y - x), \\ \dot{y} = \rho x - y - xz, \\ \dot{z} = xy - \beta z, \end{cases} \tag{6.2}$$

где σ, ρ и β — параметры системы (некоторые положительные числа, обычно указывают $\sigma = 10$, $\rho = 28$ и $\beta = 8/3$).

Рис. 2.5: Система Лоренца

Система получена из системы уравнений Навье–Стокса и описывает движение воздушных потоков в плоском слое жидкости постоянной толщины при разложении скорости течения и температуры в двойные ряды Фурье с последующем усечением до первых-вторых гармоник. Решение системы неустойчиво

на аттракторе, что не позволяет применять классические численные методы на больших отрезках времени, требуется использовать высокоточные вычисления. Численное решение в Julia будет иметь следующий вид:

```
[6]: # подключаем необходимые пакеты:
     import Pkg
     Pkg.add("DifferentialEquations")
     using DifferentialEquations, Plots;
     # задаём описание модели:
     function lorenz!(du,u,p,t)
     σ,ρ,β = p
     du[1] = σ*(u[2]-u[1])
     du[2] = u[1]*(ρ-u[3]) - u[2]
     du[3] = u[1]*u[2] - β*u[3]
     end
     # задаём начальное условие:
     u0 = [1.0,0.0,0.0]
     # задаём значения параметров:
     p = (10,28,8/3)
     # задаём интервал времени:
     tspan = (0.0,100.0)
     # решение:
     prob = ODEProblem(lorenz!,u0,tspan,p)
     sol = solve(prob)

        Resolving package versions...
          Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
         Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`

[6]: retcode: Success
     Interpolation: 3rd order Hermite
     t: 1243-element Vector{Float64}:
        0.0
        3.5678604836301404e-5
        0.0013605138799281007
        0.0058429948478927385
        0.012740296070722901
        0.021714423837051536
        0.034037716267449586
        0.04999135835114553
        0.07064197144628097
        0.09680156042982756
        0.12950796542331358
        0.1661722841482902
        0.2063449729846054
```

Рис. 2.6: Численное решение системы Лоренца

```
[7]: # подключаем необходимые пакеты:
Pkg.add("Plots")
using Plots
# строим график:
plot(sol, vars=(1,2,3), lw=2, title="Аттрактор Лоренца", xaxis="x",yaxis="y", zaxis="z",legend=false)
```

```
   Resolving package versions...
     Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.tom
    Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.to
┌ Warning: To maintain consistency with solution indexing, keyword argument vars will be removed in a f
│   caller = ip:0x0
└ @ Core :-1
```

[7]:

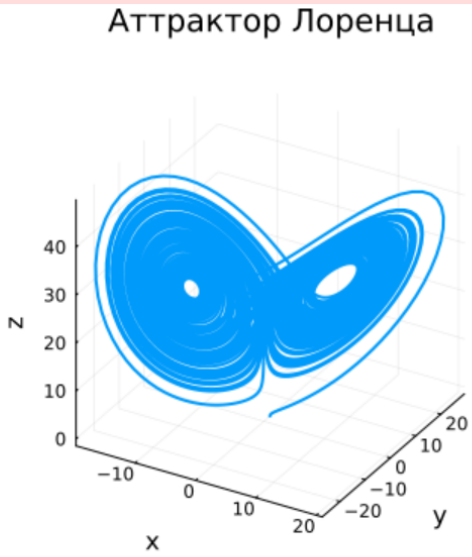

Рис. 2.7: График: Аттрактор Лоренца

```
[8]: # отключаем интерполяцию:
plot(sol,vars=(1,2,3),denseplot=false, lw=1, title="Аттрактор Лоренца", xaxis="x",yaxis="y", zaxis="z",legend=false)
```

[8]:

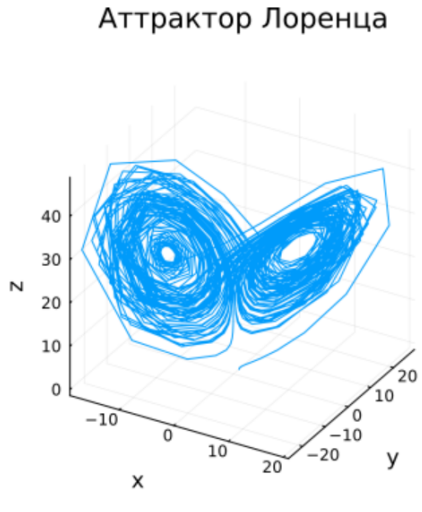

Рис. 2.8: График: Аттрактор Лоренца (интерполяция отключена)

2.2 Модель Лотки–Вольтерры

Модель Лотки–Вольтерры описывает взаимодействие двух видов типа «хищник – жертва»:

$$\begin{cases} \dot{x} = (\alpha - \beta y)x, \\ \dot{y} = (-\gamma + \delta x)y, \end{cases}$$

где x — количество жертв, y — количество хищников, t — время, $\alpha, \beta, \gamma, \delta$ — коэффициенты, отражающие взаимодействия между видами (в данном случае α — коэффициент рождаемости жертв, γ — коэффициент убыли хищников, β — коэффициент убыли жертв в результате взаимодействия с хищниками, δ — коэффициент роста численности хищников).

Рис. 2.9: Модель Лотки–Вольтерры

Численное решение в Julia будет иметь следующий вид:

```
[17]: # подключаем необходимые пакеты:
import Pkg
Pkg.add("ParameterizedFunctions")
using ParameterizedFunctions, DifferentialEquations, Plots;
# задаём описание модели:
lv! = @ode_def LotkaVolterra begin
dx = a*x - b*x*y
dy = -c*y + d*x*y
end a b c d
# задаём начальное условие:
u0 = [1.0,1.0]
# задаём знанчения параметров:
p = (1.5,1.0,3.0,1.0)
# задаём интервал времени:
tspan = (0.0,10.0)
# решение:
prob = ODEProblem(lv!,u0,tspan,p)
sol = solve(prob)
plot(sol, label = ["Жертвы" "Хищники"], color="black", ls=[:solid, :dash], title="Модель Лотки - Вольтерры", xaxis="Время",yaxis="Размер популяции")

  Resolving package versions...
    Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
   Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
```

Рис. 2.10: Численное решение модели Лотки–Вольтерры

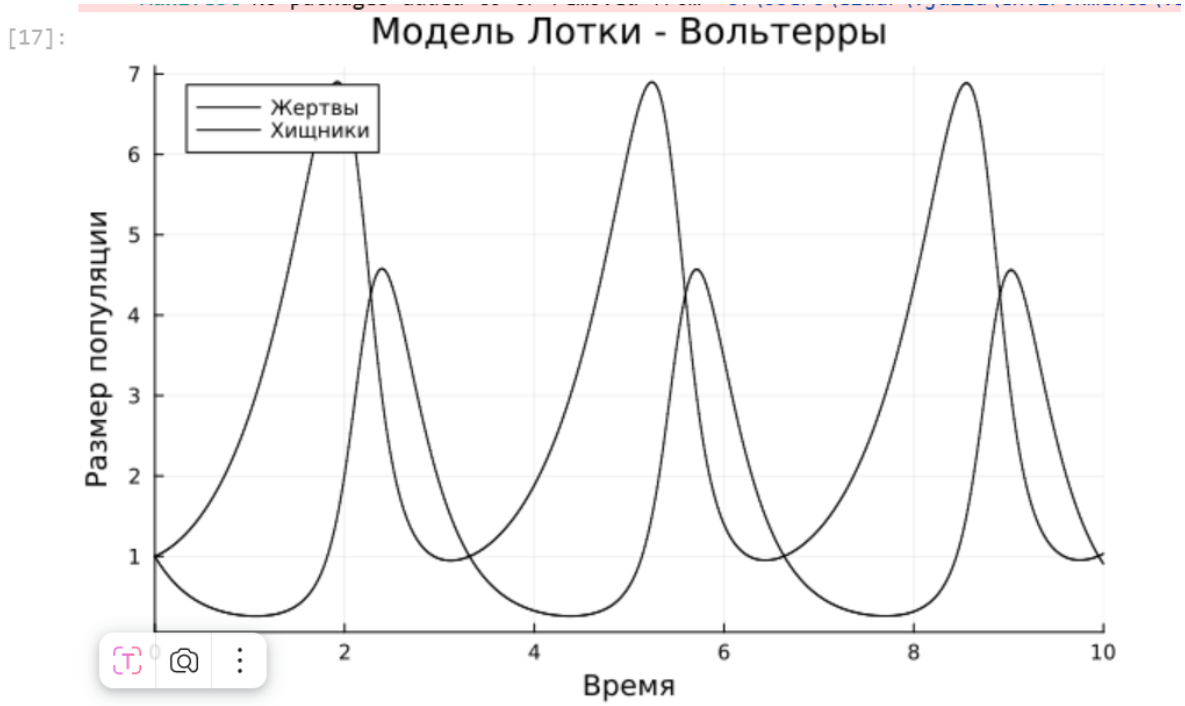

Рис. 2.11: Модель Лотки–Вольтерры: динамика изменения численности популяций

```
[18]: # фазовый портрет:
plot(sol,vars=(1,2), color="black", xaxis="Жертвы",yaxis="Хищники", legend=false)
```

[18]:

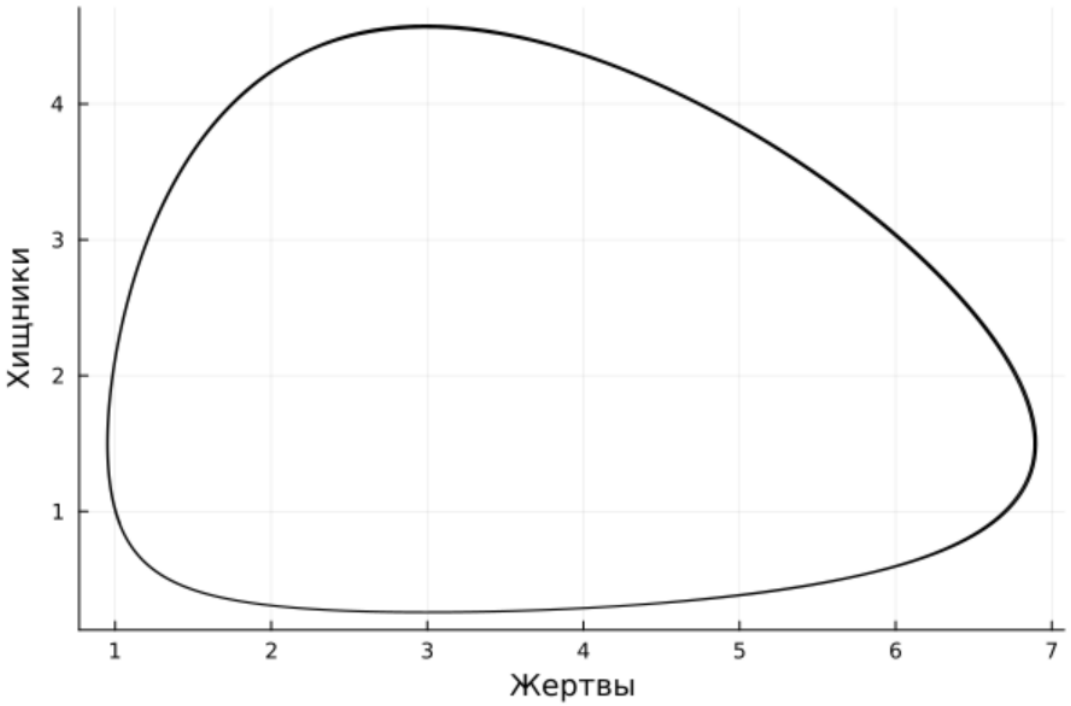

Рис. 2.12: Модель Лотки–Вольтерры: фазовый портрет

2.3 Задания для самостоятельного выполнения

1.

```
[19]: using DifferentialEquations, Plots

# Параметры модели
b = 0.05  # коэффициент рождаемости
c = 0.02  # коэффициент смертности
a = b - c # коэффициент роста популяции
x0 = 100  # начальная численность популяции
tspan = (0.0, 50.0) # интервал времени

# Функция правой части дифференциального уравнения
function maltus_model(dx, x, p, t)
    dx[1] = a * x[1]
end

# Создание задачи и её решение
u0 = [x0]
prob = ODEProblem(maltus_model, u0, tspan)
sol = solve(prob, Tsit5(), reltol=1e-8, abstol=1e-8)

# Построение графика
plot(sol, linewidth=3, xlabel="Время (t)", ylabel="Численность популяции (x(t))",
     title="Модель Мальтуса: рост изолированной популяции")
```

Рис. 2.13: Модель роста численности изолированной популяции (модель Мальтуса)

[19]:

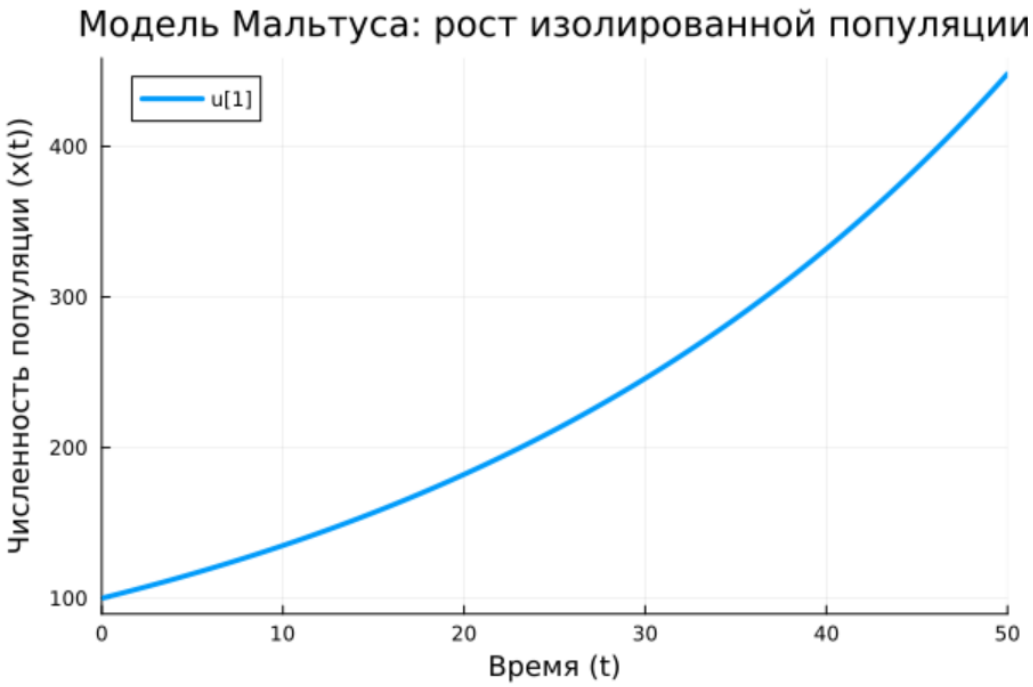

Рис. 2.14: Модель Мальтуса: график

2.

[21]:

```
using DifferentialEquations, Plots

# Параметры модели
r = 0.5  # коэффициент роста популяции
k = 500  # предельная численность (ёмкость среды)
x0 = 50  # начальная численность популяции
tspan = (0.0, 50.0) # интервал времени

# Функция правой части дифференциального уравнения
function logistic_model(dx, x, p, t)
    r, k = p
    dx[1] = r * x[1] * (1 - x[1] / k)
end

u0 = [x0]
p = [r, k]
prob = ODEProblem(logistic_model, u0, tspan, p)
sol = solve(prob, Tsit5(), reltol=1e-8, abstol=1e-8)
plot(sol, linewidth=3, xlabel="Время (t)", ylabel="Численность популяции (x(t))", title="Логистическая модель: рост с огран. ёмкости среды")
```

Рис. 2.15: Логистическая модель роста популяции

[21]:

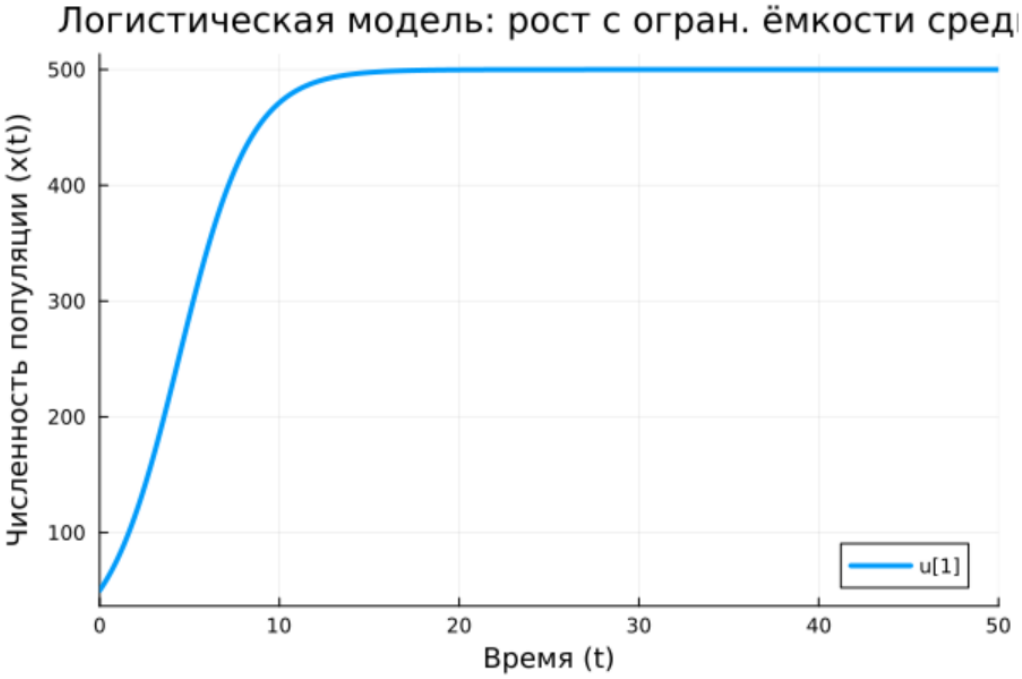

Рис. 2.16: Логистическая модель роста популяции: график

3.

```
[22]: using DifferentialEquations, Plots

# Параметры модели
β = 0.3  # коэффициент заражения (интенсивность контактов)
v = 0.1  # коэффициент выздоровления
N = 1000 # общая численность популяции
S0 = N - 10  # восприимчивые
I0 = 10      # инфицированные
R0 = 0      # переболевшие
u0 = [S0, I0, R0]

tspan = (0.0, 100.0)
# Функция правой части системы дифференциальных уравнений
function sir_model(du, u, p, t)
    β, v = p
    S, I, R = u
    N = S + I + R

    du[1] = -β * S * I / N # изменение числа восприимчивых
    du[2] = β * S * I / N - v * I # изменение числа инфицированных
    du[3] = v * I # изменение числа переболевших
end

p = [β, v]
prob = ODEProblem(sir_model, u0, tspan, p)
sol = solve(prob, Tsit5(), reltol=1e-8, abstol=1e-8)

plot(sol, linewidth=3, xlabel="Время (t)",
     ylabel="Численность групп",
     title="SIR-модель: динамика эпидемии",
     labels=["Восприимчивые (S)" "Инфицированные (I)" "Переболевшие (R)"])
```

Рис. 2.17: Модель эпидемии Кермака–Маккендрика (SIR-модель)

[22]:

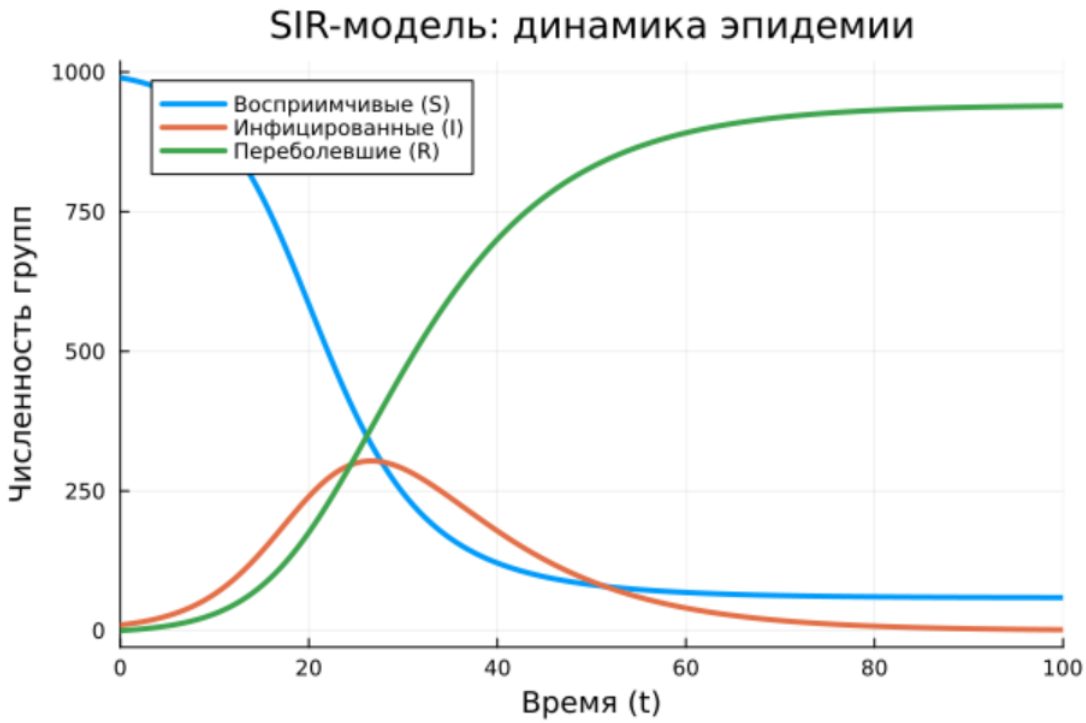

Рис. 2.18: SIR-модель: график

3 Выводы

В результате выполнения лабораторной работы, я освоил специализированные пакеты для решения задач в непрерывном и дискретном времени.